Amazon ECS & ECR & EKS Console Dashboard Wise Important Interview Questions & Answers

Amazon ECS
│
├── Dashboard
│
├── Clusters
│ └── Individual Cluster
│ ├── Services
│ ├── Tasks
│ ├── Container Instances
│ ├── Capacity Providers
│ └── Configuration
│
├── Task Definitions
│ └── Task Definition Revision
│ ├── Containers
│ ├── CPU / Memory
│ ├── Port Mappings
│ ├── Environment Variables
│ ├── IAM Roles
│ ├── Logging
│ └── Storage
│
├── Task Definitions
├── Clusters
├── Services
├── Tasks
├── Container Instances
├── Capacity Providers
└── Account Settings

1. What is Amazon ECS?
Answer:
Amazon ECS stands for Elastic Container Service and is used to run and manage Docker containers on AWS.
It manages container deployment, scheduling, networking, and scaling.
ECS can run containers using either EC2 instances or AWS Fargate.
For example:
Suppose we have a web application packaged as a Docker container.
We can create an ECS cluster and run the container as an ECS service.
ECS manages the containers while users access the application through a Load Balancer.
Example:
User
↓
Load Balancer
↓
ECS Service
↓
Docker Container
Simple ga gurthupettuko:
ECS = Run and manage containers on AWS

---

2. What is an ECS Cluster?
Answer:
An ECS Cluster is a logical grouping of resources used to run ECS tasks and services.
It provides a place where we deploy and manage our containers.
A cluster can use either EC2 capacity or Fargate capacity.
For example:
Suppose our company has a shopping application.
We create an ECS cluster called shopping-cluster and run the frontend and backend containers inside it.
The cluster becomes the logical home for those ECS workloads.
Example:
ECS Cluster
↓

↓ ↓
Frontend Backend
Container Container
Simple ga gurthupettuko:
Cluster = Logical place where ECS workloads run

---

3. What is an ECS Task Definition?
Answer:
A Task Definition is a blueprint that tells ECS how a container should run.
It defines information such as the container image, CPU, memory, ports, environment variables, IAM roles, and logging.
ECS uses the Task Definition to launch tasks.
For example:
Suppose our application uses an Nginx container.
We create a Task Definition specifying the Nginx image, CPU, memory, and port 80.
ECS uses this definition whenever it launches the container.
Example:
Task Definition
↓
Container Image
CPU + Memory
Port
Environment
Logging
↓
ECS Task
Simple ga gurthupettuko:
Task Definition = Blueprint for running a container

---

4. What is an ECS Task?
Answer:
An ECS Task is a running instance of a Task Definition.
When ECS launches a task, it creates the actual container workload according to the Task Definition.
Multiple tasks can be launched from the same Task Definition.
For example:
Suppose our Task Definition describes a web application container.
If we run three copies of that definition, ECS creates three running tasks.
This allows the application to handle more traffic.
Example:
Task Definition
↓

↓ ↓ ↓
Task-1 Task-2 Task-3
Simple ga gurthupettuko:
Task = Running copy of Task Definition

---

5. What is an ECS Service?
Answer:
An ECS Service keeps a desired number of tasks running continuously.
If a task stops unexpectedly, the service can launch another task to maintain the desired count.
Services are commonly used for long-running applications such as web servers and APIs.
For example:
Suppose we want three backend containers running all the time.
We create an ECS Service with desired count 3.
If one container fails, ECS starts another task to maintain three running tasks.
Example:
ECS Service
Desired Count = 3
↓
Task-1 Task-2 Task-3
↓
Task fails
↓
New Task starts
Simple ga gurthupettuko:
Service = Keep required number of tasks running

---

6. What is Fargate in ECS?
Answer:
AWS Fargate is a serverless compute engine for ECS containers.
With Fargate, we do not need to create or manage EC2 instances for running containers.
AWS manages the underlying infrastructure while we specify CPU, memory, networking, and container configuration.
For example:
Suppose we want to run a Docker application but do not want to manage EC2 servers.
We select Fargate as the ECS launch type or capacity provider.
AWS runs the container without us managing the underlying servers.
Example:

ECS
↓
Fargate
↓
Container
↓
Application
Simple ga gurthupettuko:
Fargate = Run containers without managing EC2 servers

---

7. What is ECS on EC2?
Answer:
ECS on EC2 means ECS tasks run on EC2 instances that we manage.
We are responsible for the EC2 instances, including their capacity, operating system, and scaling.
ECS manages the container scheduling and deployment on those instances.
For example:
Suppose we need more control over the underlying servers.
We can create an ECS cluster with EC2 instances and run our containers on them.
ECS schedules the tasks onto the available EC2 capacity.
Example:
ECS Cluster
↓
EC2 Instances
↓ ↓
Task Task
Simple ga gurthupettuko:
ECS EC2 = Containers run on our EC2 servers

---

8. What is a Capacity Provider?
Answer:
A Capacity Provider tells ECS how tasks should use available compute capacity.
Common capacity providers include Fargate and EC2 Auto Scaling Group-based capacity providers.
They help ECS manage where and how tasks are placed.
For example:
Suppose our ECS service uses Fargate for running containers.
We can configure a Fargate capacity provider for the service.
ECS then uses that capacity to launch the required tasks.
Example:
ECS Service
↓
Capacity Provider
↓
Fargate / EC2
↓
Tasks
Simple ga gurthupettuko:
Capacity Provider = Defines where ECS gets compute capacity

---

9. What is Port Mapping in ECS?
Answer:
Port Mapping connects a container port to the networking configuration used to reach the application.
It tells ECS which port the container listens on and can be used with load balancer configuration.
For example, a web application may listen on port 80 inside the container.
For example:
Suppose our Docker application listens on port 8080.
We define the container port in the Task Definition.
The ECS networking and Load Balancer configuration can then route traffic to that container port.
Example:
Load Balancer
↓
ECS Service
↓
Container
↓
Port 8080
Simple ga gurthupettuko:
Port Mapping = Connect traffic to the container port

---

10. How does ECS work with an Application Load Balancer?
Answer:
An Application Load Balancer can distribute incoming traffic across ECS tasks.

The ECS Service registers the running tasks with the target group associated with the Load Balancer.
The ALB sends user requests to healthy containers.
For example:
Suppose our ECS service has three web containers.
The ALB receives user requests and distributes them among the healthy containers.
If one task becomes unhealthy, traffic can be sent to the remaining healthy tasks.
Example:
User
↓
ALB
↓

↓ ↓ ↓
Task1 Task2 Task3
Simple ga gurthupettuko:
ALB + ECS = Distribute traffic across containers

Amazon ECR — Console Dashboard Wise Important Interview Questions & Answers ⭐⭐

Amazon ECR
│
├── Dashboard
│
├── Private Registry
│ └── Repositories
│ └── Individual Repository
│ ├── Images
│ ├── Image Tags
│ ├── Image Scan Results
│ ├── Permissions
│ └── Lifecycle Policy
│
├── Repositories
│
├── Public Registry
│
├── Private Registry
│
└── Repository Settings
├── Permissions
├── Lifecycle Policy
└── Encryption

11. What is Amazon ECR?
Answer:
Amazon ECR stands for Elastic Container Registry and is a managed container image registry.
It allows us to store, manage, and retrieve Docker or OCI container images.
ECR integrates directly with ECS, EKS, and other AWS services.
For example:
Suppose we build a Docker image for our application.
We push that image into an ECR repository.
ECS or EKS can then pull the image from ECR and run the application.
Example:
Developer
↓
Docker Image
↓
ECR Repository
↓
ECS / EKS
Simple ga gurthupettuko:
ECR = Store Docker/Container Images

---

12. What is an ECR Repository?
Answer:
An ECR Repository is a location where container images are stored.
Each repository can contain multiple image versions identified by tags or image digests.
We normally create separate repositories for different applications or services.
For example:
Suppose we have a frontend application and a backend application.

We can create two ECR repositories, one for each application.
Each repository can contain multiple versions of its container image.
Example:
ECR
↓
Repositories
├── frontend
│ ├── v1
│ └── v2
└── backend
├── v1
└── v2
Simple ga gurthupettuko:
Repository = Place where container images are stored

---

13. What is an ECR Image Tag?
Answer:
An image tag is a human-readable label used to identify a particular version or build of a container image.
Common tags include latest, v1, v2, or environment-specific names.
Tags make it easier to select which image version should be deployed.
For example:
Suppose we build three versions of our application.
We can tag them as v1, v2, and v3 and push them to ECR.
ECS can then be configured to use the required image version.
Example:
ECR Repository
↓
app:v1
app:v2
app:v3
Simple ga gurthupettuko:
Tag = Label used to identify an image version

---

14. What is ECR Image Scanning?
Answer:
ECR image scanning checks container images for known security vulnerabilities.
The scan results help us identify vulnerable packages inside the image.
We can use the results to update the image and deploy a safer version.
For example:
Suppose we push a Docker image containing an outdated operating system package.
ECR scanning can identify known vulnerabilities associated with that package.
We can rebuild the image with an updated package and push the new version.
Example:
Docker Image
↓
ECR
↓
Image Scan
↓
Vulnerabilities
↓
Fix + Rebuild
Simple ga gurthupettuko:
ECR Scan = Check container image for vulnerabilities

---

15. What is an ECR Lifecycle Policy?
Answer:
An ECR Lifecycle Policy automatically manages old container images in a repository.
It can expire images based on rules such as image age or number of images retained.
This helps reduce storage usage and keeps repositories clean.
For example:
Suppose our CI/CD pipeline pushes a new image every day.
After several months, the repository may contain hundreds of old images.
A lifecycle policy can automatically remove old images that are no longer required.
Example:
ECR Repository
↓
100 Images
↓
Lifecycle Policy
↓

Keep Recent Images
↓
Remove Old Images
Simple ga gurthupettuko:
Lifecycle Policy = Automatically clean old images

---

16. How does ECS/EKS pull an image from ECR?
Answer:
The ECS task or EKS workload needs permission to access the ECR repository.
The workload authenticates with ECR and pulls the required container image.
The container runtime then uses that image to start the application.
For example:
Suppose an ECS task definition specifies an image stored in ECR.
When ECS launches the task, the required AWS permissions allow it to retrieve the image.
ECS then starts the container using that image.
Example:
ECS / EKS
↓
IAM Permission
↓
ECR Repository
↓
Container Image
↓
Running Container
Simple ga gurthupettuko:
ECR = Image Store | ECS/EKS = Run the Image

Amazon EKS — Console Dashboard Wise Important Interview Questions & Answers ⭐ ⭐

Amazon EKS
│
├── Dashboard
│
├── Clusters
│ └── Individual Cluster
│ ├── Overview
│ ├── Compute
│ ├── Add-ons
│ ├── Networking
│ ├── Configuration
│ ├── Logging
│ └── Updates
│
├── Clusters
├── Add-ons
├── Node Groups
├── Pod Identity
└── Access

17. What is Amazon EKS?
Answer:
Amazon EKS stands for Elastic Kubernetes Service and is a managed Kubernetes service on AWS.
It allows us to deploy, manage, and scale containerized applications using Kubernetes.
AWS manages the Kubernetes control plane, while we manage the workloads and compute according to the chosen architecture.
For example:
Suppose our company already uses Kubernetes for container management.
Instead of building the Kubernetes control plane ourselves, we can create an EKS cluster.
We can then deploy our applications using Kubernetes resources such as Pods and Deployments.
Example:
AWS
↓
EKS Cluster
↓
Kubernetes
↓
Pods
↓
Containers
Simple ga gurthupettuko:
EKS = Managed Kubernetes on AWS

18. What is an EKS Cluster?
Answer:
An EKS Cluster is the main Kubernetes environment where our workloads run.
It contains the Kubernetes control plane managed by AWS and compute resources where application Pods run.
We use the cluster to deploy and manage Kubernetes applications.
For example:
Suppose we have frontend and backend applications.
We create one EKS cluster and deploy the frontend and backend Pods into it.
Kubernetes manages those workloads according to their configuration.
Example:
EKS Cluster
↓

↓ ↓
Frontend Pods Backend Pods
Simple ga gurthupettuko:
EKS Cluster = Kubernetes environment

---

19. What are EKS Node Groups?
Answer:
Node Groups are groups of worker nodes used to run Kubernetes Pods.
EKS supports managed node groups where AWS helps manage the underlying EC2 instances.
We can configure the instance type, scaling, subnets, and other node settings.
For example:
Suppose our application needs EC2 worker nodes to run Pods.
We create an EKS managed node group with several EC2 instances.
Kubernetes can then schedule application Pods onto those worker nodes.
Example:
EKS Cluster
↓
Managed Node Group
↓ ↓
EC2-1 EC2-2
↓ ↓
Pods Pods
Simple ga gurthupettuko:
Node Group = Worker machines that run Kubernetes Pods

---

20. What are EKS Add-ons?
Answer:
EKS Add-ons provide supported Kubernetes components that extend the functionality of an EKS cluster.
Examples include networking, DNS, and observability-related components.
EKS can manage compatible add-on versions and their lifecycle.
For example:
Suppose our EKS cluster needs the VPC CNI component for Kubernetes networking.
We can configure it as an EKS add-on.
EKS helps manage the supported version of that component.
Example:
EKS Cluster
↓
Add-ons
├— VPC CNI
├— CoreDNS
└— kube-proxy
Simple ga gurthupettuko:
EKS Add-ons = Important Kubernetes components managed with EKS

---

21. What is EKS Networking?
Answer:
EKS networking connects Kubernetes Pods, nodes, and AWS services through the VPC.
EKS commonly uses the Amazon VPC CNI plugin to provide networking for Pods.
We configure subnets, security groups, and other networking settings according to the cluster design.
For example:
Suppose our EKS cluster is deployed across multiple subnets.
The worker nodes and Pods use the VPC networking configuration to communicate with other AWS resources.
Security groups and network controls can restrict unwanted traffic.
Example:
VPC
↓
EKS

↓
Subnets
↓
Nodes
↓
Pods
↓
AWS Services
Simple ga gurthupettuko:
EKS Networking = Connect Pods and nodes through VPC

---

22. What is EKS Logging?
Answer:
EKS can send supported Kubernetes control-plane logs to Amazon CloudWatch Logs.
These logs help us troubleshoot cluster activity and understand control-plane operations.
We enable the required log types depending on our monitoring and troubleshooting needs.
For example:
Suppose we are troubleshooting an issue with our Kubernetes cluster.
We enable the required EKS control-plane logging and send the logs to CloudWatch.
The operations team can then review those logs for troubleshooting.
Example:
EKS Cluster
↓
Control Plane Logs
↓
CloudWatch Logs
↓
Troubleshooting
Simple ga gurthupettuko:
EKS Logging = Send cluster logs to CloudWatch

---

23. What is EKS Pod Identity?
Answer:
EKS Pod Identity allows Kubernetes Pods to obtain AWS permissions without giving those permissions directly to the worker node.
It helps applications securely access AWS services using IAM permissions.
This supports least-privilege access for workloads.
For example:
Suppose a Kubernetes application needs to read objects from an S3 bucket.
We can configure Pod Identity so that only the required Pod gets the necessary AWS permissions.
The application can then access S3 without giving every node that permission.
Example:
Kubernetes Pod
↓
EKS Pod Identity
↓
IAM Permissions
↓
S3
Simple ga gurthupettuko:
Pod Identity = Give AWS permissions to specific Kubernetes Pods

---

24. ECS vs EKS — What is the main difference?
Answer:
ECS is AWS's container orchestration service, while EKS is AWS's managed Kubernetes service.
ECS is generally simpler when we want an AWS-native container platform without managing Kubernetes concepts.
EKS is useful when the organization needs Kubernetes compatibility and its ecosystem.
For example:
Suppose our team wants a simple AWS container platform.
ECS with Fargate can be a straightforward choice because we do not need to manage Kubernetes.
If our team already uses Kubernetes, EKS can provide managed Kubernetes on AWS.
Example:
ECS
↓
AWS Container Orchestration

EKS
↓
Kubernetes
↓
Containers **Simple ga gurthupettuko:**
ECS = AWS-native containers | EKS = Kubernetes on AWS